ENGENHARIA DE SOFTWARE / SPSKILLS

Negócio e Problema

Entenda o que precisa ser resolvido antes de começar a construir.

A pergunta que orienta tudo:

“O que precisa funcionar no fim?”

Leitura de requisitos • regras • solução

Construir certo começa por entender o problema

“Durante a competição, o erro mais comum não é a falta de habilidade, e sim construir da maneira errada.”

Antes do código, transforme o enunciado em decisões claras.

Três perguntas reduzem o risco de errar o alvo

Responda antes de escolher telas, arquitetura ou tecnologia.



01 **Qual dor real o sistema resolve?**

PROBLEMA DE NEGÓCIO

02 **Quem usa e quais regras são obrigatórias?**

ESCOPO FUNCIONAL

03 **O que valida o sucesso?**

CRITÉRIO DE SUCESSO

Comece pela dor que justifica o sistema

O motivo pelo qual a aplicação precisa existir.

Exemplo

“Preciso saber quanto tempo estou treinando para a olimpíada da SPSKILLS.”

Dor identificada

Falta de acompanhamento do tempo de treino

Da dor à necessidade

Acompanhar
→ registrar sessões
→ somar duração
→ consultar histórico

Elegibilidade depende de todas as regras

Exemplo ilustrativo: aceitar um novo membro para a Liga da Justiça.



E



ACEITO

Da DC + herói

RECUSADO

Herói, mas não é da DC

Defina como reconhecer que está funcionando

Critério de sucesso

O que valida o sistema e atende aos critérios da prova.

Requisito

“O sistema registra o tempo de treino.”

Evidência de sucesso

Uma sessão pode ser registrada e sua duração aparece no histórico.

Critério claro → teste objetivo → menos interpretação na hora da entrega

Separe o enunciado em três blocos

01 **CORE BUSINESS**

O que faz o sistema ter utilidade?

Banco: entrada e saída de dinheiro; cadastro de clientes.

02 **REGRAS CRÍTICAS**

O que deve ser validado no backend e nas permissões?

Marque “deve”, “não pode”, “somente se” e “caso contrário”.

03 **FLUXO DE VALOR**

Como o dado entra, é processado e vira resultado?

Entrada → processamento → entrega de valor.

Termos absolutos sinalizam regras críticas



DEVE

NÃO PODE

SOMENTE SE

CASO CONTRÁRIO

Leia com atenção: esses termos costumam virar validação ou permissão.

A melhor solução é a mais simples que atende tudo

“Busque o melhor da simplicidade.”

ARMADILHA

Arquitetura pesada para um problema que pede uma aplicação modular.

CAMINHO ADEQUADO

Escolha a estrutura mais simples que cumpra 100% dos requisitos e seja fácil de manter.

Reaproveite o que já resolve mais de uma parte do problema.

Construa em etapas que protegem o essencial

1

MVP

O mínimo aceitável que serve de base para as próximas etapas.

2

Domínio

Entidades, dados e relacionamentos que representam o problema.

3

Implementação

Uma funcionalidade por vez, seguindo as regras.

Entenda → modele → implemente → valide